paiement-api — SEPA transfer microservice — RTS 2024/1774 · Internal policy PDS-2 (Banque Kerlann, fictitious)

Banque Kerlann (fictitious) · paiement-api — SEPA transfer microservice

Prepared by: **Attest (AI-assisted)**

Auditor sign-off: **pending**

Owner: **IT department — Payments development team**

Date: **2026-11-04 10:25**

Run: **20261104-102505-7a3f**

1. Executive summary

OPINION AT TESTING

Unsatisfactory

Basis for opinion

7 controls tested (7 conclusive) against: RTS 2024/1774 — Article 16: ICT systems acquisition, development and maintenance · Internal policy PDS-2 (Banque Kerlann, fictitious) — secure development. Tests run on 2026-11-04 10:25, environment: Demonstration pre-production. The opinion rests solely on the verdicts of the deterministic test scripts and on the evidence sealed in the audit trail.

Scope limitation: Work programme and test scripts drafted by AI and not yet reviewed by an auditor; requirements needing document review or interview are not covered; the opinion applies at the test date only.

7 controls tested	**7** findings	**0** findings closed and verified	**7** open findings
0 human approvals	**18** sensitive data items masked in evidence	**3 min 12 s** audit duration	**$0.546 *** measured AI cost



Critical: 1 High: 3 Medium: 3 Low: 0

Key messages

- 7 findings across 7 controls: 1 critical, 3 high, 3 medium.
- 7 findings remain open, with an action plan, an owner and a due date.

2. Scope and method

Engagement	AUD-20261104-7A3F
Entity	Banque Kerlann (fictitious)
System	paiement-api — SEPA transfer microservice
System criticality	Supports critical or important functions (DORA)
Environment	Demonstration pre-production
Owner	IT department — Payments development team
Lead auditor	Attest CI

Date	2026-11-04 10:25
Framework	RTS 2024/1774 — Article 16: ICT systems acquisition, development and maintenance · Internal policy PDS-2 (Banque Kerlann, fictitious) — secure development
Audited target	paiement-api code repository (SEPA transfer microservice)
Work programme	existing catalogue reused · runs/studio/20261104-102505-7a3f/catalogue.json

In scope

- Source code, configuration and CI pipeline of the repository
- Versioned test datasets

Out of scope

- Dynamic testing (DAST) and penetration testing
- Runtime and production environments
- Git history

Method

1. Controls are derived from the framework (DORA, RTS 2024/1774, internal policy); severity is the one set by the policy.
2. Each test is a stored Python script, statically checked before execution (read-only) and run with a read-only service account.
3. The verdict comes from the script, never from the model. A screenshot is reviewed by Muse Spark to corroborate; on disagreement the control is flagged for review.
4. SAM 3.1 delineates the proving rows and the personal data, which is masked before distribution.
5. Every fix goes through a change request approved by the system owner, applied with a dedicated account, logged, then verified by the same test (DORA 9(4)(e)).
6. Every event and piece of evidence is sealed in a hash-chained audit trail (SHA-256).

Limitations

- Tests cover the configuration reported by the system, not the actual strength of authentication factors.
- AI-drafted scripts and catalogue must be reviewed by an auditor before being considered validated.
- Fixes are drafted by the audit tool itself (self-review threat): closing a finding requires independent approval by the system owner.
- Fictitious entity and data: demonstration.

3. Findings overview

REF.	RULE	TITLE	SEVERITY	TESTED	AFTER	STATUS	DUE DATE
F-01	PDS-2.1	No hardcoded secrets in code or versioned config	Critical	Non-compliant		Open	2026-12-04
F-02	PDS-2.2	TLS certificate verification never disabled	High	Non-compliant		Open	2027-02-02
F-03	PDS-2.3	Approved password hashing only	High	Non-compliant		Open	2027-02-02
F-04	PDS-2.6	Synthetic test data only	High	Non-compliant		Open	2027-02-02
F-05	PDS-2.4	CI runs static security analysis on every change	Medium	Non-compliant		Open	2027-05-03
F-06	PDS-2.5	Pinned dependencies audited in CI	Medium	Non-compliant		Open	2027-05-03
F-07	PDS-2.7	Authentication successes and failures logged	Medium	Non-compliant		Open	2027-05-03

4. Detailed findings and controls

F-01 **PDS-2.1** **No hardcoded secrets in code or versioned config** — Critical — Non-compliant — Open

CRITERIA

Verify no password, key or token is written in clear text in source or versioned config.

PDS-2.1 · RTS 7(2)

Threshold source: PDS-2.1

> *« Aucun secret (mot de passe, clé, jeton) n'est écrit en clair dans le code source ni dans la configuration versionnée : les secrets viennent d'un coffre ou de variables d'environnement. »*

CONDITION

Observed: `{"hardcoded_count": 2, "findings": [{"path": "app/config.py", "line": 6, "text": "DB_PASSWORD = \"Az••••\" # TODO : déplacer dans le coffre-fort"}, {"path": "a…`

Expected: `Zero hardcoded secret assignments; all secrets obtained from vault or environment variables.`

Found 2 hardcoded secret assignments in reviewed files

PROBABLE CAUSE

Developer convenience hardcoding credentials for local testing. (AI hypothesis, not tested: to be confirmed with the owner)

RISK

Credential theft and unauthorized access to production and payment data.

RECOMMENDATION

Remove hardcoded secrets and load them from a vault or environment variables.

Due date: 2026-12-04 · Owner: IT department — Payments development team

Test procedure: 1. GET /api/repo/paiement-api/tree to list files. 2. GET /api/repo/paiement-api/search?q=(?i)(password|passwd|secret|api[_-]?key|aws_secret|token)\s*[:=]\s*['"][^'"]+ with path_prefix app/ and without prefix. 3. GET /api/repo/paiement-api/file?path=app/config.py and any matched files to inspect context. 4. Verify secret access uses os.environ, os.getenv or vault client and no clear-text value.

Banque Kerlann · dépôts de code paiement-api

app/config.py

```
1  """Configuration du service."""
2  import os
3
4  PSP_URL = os.getenv("PSP_URL", "https://psp.kerlann-banque.example/v1/transfers")
5  DB_URL = "postgresql://[redacted]:5432/paiement"
6  DB_PASSWORD = '[redacted]'  # TODO : déplacer dans le coffre-fort
7  JWT_SECRET = '[redacted]'
8  TIMEOUT_S = 10
```

Dépôt de démonstration — entreprise et données fictives.

At testing — The screenshot corroborates the test. The view shows hardcoded DB_PASSWORD and JWT_SECRET assignments on lines 6 and 7 matching the automated findings. · 3/3 sensitive data items masked (0 spotted by the AI, the rest by the deterministic safety net)
`2026-11-04 10:26:22 · SHA-256 c9cdc3fffafbeda4…`

F-02 PDS-2.2 TLS certificate verification never disabled

High | Non-compliant | Open

CRITERIA

Verify no outgoing call disables TLS server certificate verification.

PDS-2.2 · DORA 9(3)(a) ; RTS 6(2)(a)

Threshold source: PDS-2.2

> *« Tout appel réseau sortant vérifie le certificat TLS du serveur : la vérification n'est jamais désactivée (`verify=False` interdit). »*

CONDITION

Observed: `1`

Expected: `Zero matches for verify=False or equivalent TLS verification bypass.`

Found 1 matches disabling TLS verification confirmed in app/payments.py.

PROBABLE CAUSE

Disabled verification to allow self-signed certificates in test. (AI hypothesis, not tested: to be confirmed with the owner)

RISK

Man-in-the-middle interception of SEPA transfer data.

RECOMMENDATION

Enforce TLS verification on all outgoing calls and remove any verify=False.

Due date: 2027-02-02 · Owner: IT department — Payments development team

Test procedure: 1. GET /api/repo/paiement-api/search?q=verify\s*=\s*False. 2. GET /api/repo/paiement-api/search?q=ssl\._create_unverified_context|CERT_NONE|check_hostname\s*=\s*False. 3. Review matched files via GET /api/repo/paiement-api/file?path=<file> to confirm context of requests, urllib3 or http client calls.

Banque Kerlann · dépôts de code paiement-api

app/payments.py

```
1  """Initiation de virements SEPA via le prestataire de paiement (PSP)."""
2  import logging
3
4  import requests
5
6  from .config import PSP_URL, TIMEOUT_S
7
8  log = logging.getLogger("paiement")
9
10
11 def initiate_transfer(debtor_iban: str, creditor_iban: str, amount_cents: int, reference: str) -> dict:
12     payload = {
13         "debtor": {"iban": debtor_iban},
14         "creditor": {"iban": creditor_iban},
15         "amount": {"value": amount_cents, "currency": "EUR"},
16         "reference": reference,
17     }
18     response = requests.post(PSP_URL, json=payload, timeout=TIMEOUT_S, verify=False)
19     response.raise_for_status()
20     log.info("virement initié ref=%s", reference)
21     return response.json()
```

Dépôt de démonstration — entreprise et données fictives.

At testing — The screenshot corroborates the test. Line 18 in app/payments.py shows verify=False in the requests.post call.

`2026-11-04 10:26:26 · SHA-256 168099bdf34367c6…`

F-03 PDS-2.3 Approved password hashing only

High | Non-compliant | Open

CRITERIA

CONDITION

Verify passwords use argon2, bcrypt, scrypt or PBKDF2 and no MD5 or SHA-1 for security.

PDS-2.3 · RTS 6(3) et 6(4)

Threshold source: PDS-2.3

> *« Les mots de passe sont hachés avec argon2, bcrypt, scrypt ou PBKDF2. MD5 et SHA-1 sont interdits pour tout usage de sécurité. »*

Observed: `[{"line": 8, "path": "app/auth.py", "text": "return hashlib.md5(password.encode()).hexdigest()"}]`

Expected: `Password hashing uses only argon2, bcrypt, scrypt or PBKDF2; zero use of MD5 or SHA-1 for security.`

Found 1 MD5 or SHA-1 password uses requiring migration to approved hashing.

PROBABLE CAUSE

Legacy code using hashlib MD5/SHA-1 examples. (AI hypothesis, not tested: to be confirmed with the owner)

RISK

Password cracking and credential compromise due to weak hashes.

RECOMMENDATION

Migrate password hashing to argon2, bcrypt, scrypt or PBKDF2 and remove MD5/SHA-1.

Due date: 2027-02-02 · Owner: IT department — Payments development team

Test procedure: 1. GET /api/repo/paiement-api/search?q=argon2|bcrypt|scrypt|pbkdf2|passlib|hashlib. 2. GET /api/repo/paiement-api/search?q=hashlib\.(md5|sha1)\(|\.md5\(|\.sha1\(|MD5|SHA1. 3. GET /api/repo/paiement-api/file?path=<matched file> to confirm password-hashing context versus non-security use.

Banque Kerlann · dépôts de code paiement-api

app/auth.py

```
1  """Authentification des opérateurs back-office."""
2  import hashlib
3
4  from .db import get_operator
5
6
7  def hash_password(password: str) -> str:
8      return hashlib.md5(password.encode()).hexdigest()
9
10
11 def login(username: str, password: str) -> bool:
12     operator = get_operator(username)
13     if operator is None:
14         return False
15     return operator["password_hash"] == hash_password(password)
```

Dépôt de démonstration — entreprise et données fictives.

At testing — The screenshot corroborates the test. Screenshot shows app/auth.py line 8 using hashlib.md5 for password hashing.
`2026-11-04 10:26:51 · SHA-256 9b670c1eafd915cb…`

F-04 PDS-2.6 Synthetic test data only

High · Non-compliant · Open

CRITERIA

Verify test data is declared synthetic and no production export is versioned.

PDS-2.6 · RTS 16(5)(a)

Threshold source: PDS-2.6

> *« Les données de test sont synthétiques et déclarées comme telles (champ `"synthetic": true`) ; aucun export de production n'est versionné. »*

CONDITION

Observed: `{"fixture_file": "tests/fixtures/clients_export_prod.json", "synthetic_true_in_file": false, "synthetic_matches": 1, "prod_search_matches": 3, "prod_suspects": …`

Expected: `Test data files contain "synthetic": true; zero versioned production exports.`

Fixture tests/fixtures/clients_export_prod.json shows synthetic

true=False with 1 synthetic hits and 2 suspected production exports.

PROBABLE CAUSE

Copy of production data committed for realistic testing. (AI hypothesis, not tested: to be confirmed with the owner)

RISK

Leak of real customer data in non-production via versioned exports.

RECOMMENDATION

Replace test data with synthetic data marked "synthetic": true and remove production exports.

Due date: 2027-02-02 · Owner: IT department — Payments development team

Test procedure: 1. GET /api/repo/paiement-api/tree to list tests/ and fixtures. 2. GET /api/repo/paiement-api/search?q=synthetic to verify presence of "synthetic": true. 3. GET /api/repo/paiement-api/file?path=<fixture file> to confirm declaration. 4. GET /api/repo/paiement-api/search?q=prod|production.*dump|real.*iban|export.*prod to flag suspected production exports.

Banque Kerlann · dépôts de code paiement-api

tests/fixtures/clients_export_prod.json

```
{
  "export": "production",
  "date": "2026-09-30",
  "source": "base clients paiement (prod)",
  "clients": [
    {
      "nom": '[redacted]',
      "email": '[redacted]',
      "iban": '[redacted]',
      "plafond_virement_eur": 3000
    },
    {
      "nom": '[redacted]',
      "email": '[redacted]',
      "iban": '[redacted]',
      "plafond_virement_eur": 10000
    },
    {
      "nom": '[redacted]',
      "email": '[redacted]',
      "iban": '[redacted]',
      "plafond_virement_eur": 10000
    },
    {
      "nom": '[redacted]',
      "email": '[redacted]',
      "iban": '[redacted]',
      "plafond_virement_eur": 10000
    },
    {
      "nom": '[redacted]',
      "email": '[redacted]',
      "iban": '[redacted]',
      "plafond_virement_eur": 5000
    }
  ]
}
```

Dépôt de démonstration — entreprise et données fictives.

At testing — The screenshot corroborates the test. The file clients_export_prod.json declares export production from a prod source with no synthetic true visible. · 15/15 sensitive data items masked (0 spotted by the AI, the rest by the deterministic safety net)

2026-11-04 10:27:57 · SHA-256 d603feab519b9c27…

F-05 PDS-2.4 CI runs static security analysis on every change

Medium | Non-compliant | Open

CRITERIA

Verify CI executes bandit, semgrep or CodeQL on each change.

PDS-2.4 · RTS 16(3)

Threshold source: PDS-2.4

> *« La chaîne d'intégration continue exécute une analyse statique de sécurité (bandit, semgrep ou CodeQL) à chaque modification. »*

CONDITION

Observed: `{"workflows": [".github/workflows/ci.yml"], "checks": {".github/workflows/ci.yml": {"push": true, "pull_request": false, "tool": false}}, "search_matches": []}`

Expected: `A workflow triggered on push and pull_request runs bandit, semgrep or CodeQL.`

No workflow triggered on push and pull_request runs bandit, semgrep or CodeQL.

PROBABLE CAUSE

Security scanning omitted from pipeline configuration. (AI hypothesis, not tested: to be confirmed with the owner)

RISK

Undetected code vulnerabilities reaching production.

RECOMMENDATION

Add bandit, semgrep or CodeQL to CI on every push and pull request.

Due date: 2027-05-03 · Owner: IT department — Payments development team

Test procedure: 1. GET /api/repo/paiement-api/tree to list .github/workflows/. 2. GET /api/repo/paiement-api/file?path=.github/workflows/ci.yml (and any other yml) to inspect triggers and jobs. 3. GET /api/repo/paiement-api/search?q=bandit|semgrep|codeql|CodeQL to confirm tool invocation.

Banque Kerlann · dépôts de code paiement-api

.github/workflows/ci.yml

```
1  name: ci
2  on: [push]
3  jobs:
4    test:
5      runs-on: ubuntu-latest
6      steps:
7        - uses: actions/checkout@v4
8        - uses: actions/setup-python@v5
9          with:
10           python-version: "3.12"
11       - run: pip install -r requirements.txt pytest
12       - run: pytest -q
```

Dépôt de démonstration — entreprise et données fictives.

At testing — The screenshot corroborates the test. Workflow triggers only on push and runs only pytest with no bandit, semgrep or CodeQL.

2026-11-04 10:26:02 · SHA-256 bb63a9ca7ffbe74a…

F-06 PDS-2.5 Pinned dependencies audited in CI

Medium | Non-compliant | Open

CRITERIA

CONDITION

Verify exact version pinning and CI dependency audit.

PDS-2.5 · RTS 16(4) et 16(8)

Threshold source: PDS-2.5

> *« Les dépendances sont épinglées à une version exacte (`paquet==version`) et auditées en intégration continue (pip-audit ou équivalent). »*

Observed: `{"requirements_lines": ["flask", "requests>=2.0", "pyjwt==2.8.0"], "total_entries": 3, "unpinned": ["flask", "requests>=2.0"], "workflows": [".github/workflows/…`

Expected: `All requirements pinned as package==version; CI runs pip-audit or equivalent.`

Found 2 of 3 requirement entries without exact == pinning

PROBABLE CAUSE

Use of floating version ranges without vulnerability scanning. (AI hypothesis, not tested: to be confirmed with the owner)

RISK

Supply-chain compromise via unpinned or vulnerable dependencies.

RECOMMENDATION

Pin all dependencies with == and add pip-audit or equivalent to CI.

Due date: 2027-05-03 · Owner: IT department — Payments development team

Test procedure: 1. GET /api/repo/paiement-api/file?path=requirements.txt to verify every entry uses == with exact version. 2. GET /api/repo/paiement-api/tree for .github/workflows and read workflow files. 3. GET /api/repo/paiement-api/search?q=pip-audit|pip_audit|safety|osv-scanner to confirm audit step in CI.



At testing — The screenshot corroborates the test. The screenshot shows flask unpinned and requests with a >= range while only pyjwt uses exact == pinning.

`2026-11-04 10:26:54 · SHA-256 94071ff0cc3ec301…`

F-07 PDS-2.7 Authentication successes and failures logged

Medium · Non-compliant · Open

CRITERIA

Verify the application logs both successful and failed authentications.

PDS-2.7 · RTS 12(2)(c)(i)

Threshold source: PDS-2.7

> *« Les réussites et les échecs d'authentification sont journalisés par l'application. »*

CONDITION

Observed: `{"path": "app/auth.py", "log_events": [], "success_logged": false, "failure_logged": false}`

Expected: `Code contains logging for both authentication success and authentication failure events.`

app/auth.py does not emit log events for both authentication success and authentication failure.

PROBABLE CAUSE

Incomplete logging with only errors or no auth events logged. (AI hypothesis, not tested: to be confirmed with the owner)

RISK

Undetected brute-force or account takeover due to missing audit trail.

RECOMMENDATION

Add logging for both authentication successes and failures.

Due date: 2027-05-03 · Owner: IT department — Payments development team

Test procedure: 1. GET /api/repo/paiement-api/search?q=log.*auth|auth.*log|login.*(success|fail)|logger\.(info|warning|error). 2. GET /api/repo/paiement-api/file?path=<matched auth file> to confirm both success and failure branches emit a log event.

Banque Kerlann · dépôts de code paiement-api

app/auth.py

```
1  """Authentification des opérateurs back-office."""
2  import hashlib
3
4  from .db import get_operator
5
6
7  def hash_password(password: str) -> str:
8      return hashlib.md5(password.encode()).hexdigest()
9
10
11 def login(username: str, password: str) -> bool:
12     operator = get_operator(username)
13     if operator is None:
14         return False
15     return operator["password_hash"] == hash_password(password)
```

Dépôt de démonstration — entreprise et données fictives.

At testing — The screenshot corroborates the test. The login function returns on failure and success branches with no logging calls visible.
`2026-11-04 10:28:18 · SHA-256 5b3695ed7ed9f7b6…`

Appendices

A. Traceability matrix

REQUIREMENT	RULE	CONTROL	SCRIPT (SHA-256)	RESULT
RTS 7(2)	PDS-2.1	C1 · No hardcoded secrets in code or versioned config	9b8684dccf6e080f · written by AI for this engagement	Non-compliant
DORA 9(3)(a) ; RTS 6(2)(a)	PDS-2.2	C2 · TLS certificate verification never disabled	31bbad231688edb0 · written by AI for this engagement	Non-compliant
RTS 6(3) et 6(4)	PDS-2.3	C3 · Approved password hashing only	b087de888e334034 · written by AI for this engagement	Non-compliant
RTS 16(3)	PDS-2.4	C4 · CI runs static security analysis on every change	ffd8a77718a048e7 · written by AI for this engagement	Non-compliant
RTS 16(4) et 16(8)	PDS-2.5	C5 · Pinned dependencies audited in CI	cae079874abe5b60 · written by AI for this engagement	Non-compliant
RTS 16(5)(a)	PDS-2.6	C6 · Synthetic test data only	0f2117d3c55e5a68 · written by AI for this engagement	Non-compliant
RTS 12(2)(c)(i)	PDS-2.7	C7 · Authentication successes and failures logged	cfb54dd3af637813 · written by AI for this engagement	Non-compliant

B. Evidence register

FILE			SHA-256
evidence/C1_before.png	C1 · At testing	2026-11-04 10:26:22	c9cdc3fffafbeda42a2d0e534a1227e238dc765c817d5f919b43bff7d2f34742
evidence/C2_before.png	C2 · At testing	2026-11-04 10:26:26	168099bdf34367c6af05eb21777c45a42a9ffe770d00194506fe75f1de923b8b
evidence/C3_before.png	C3 · At testing	2026-11-04 10:26:51	9b670c1eafd915cb6df201edf42dc67d9cd479c769ad7b5f49bc574e294bd891
evidence/C4_before.png	C4 · At testing	2026-11-04 10:26:02	bb63a9ca7ffbe74a967dda77e5f0e0f63a2118786a6ad3fc0a6e400cd7a896e7
evidence/C5_before.png	C5 · At testing	2026-11-04 10:26:54	94071ff0cc3ec3013b5c4620d989ef9e69def04847069f8c7a03fba625a6574f
evidence/C6_before.png	C6 · At testing	2026-11-04 10:27:57	d603feab519b9c2747e18ddb985fe13abae028ac459dbb8cdb26de0b430c35b0
evidence/C7_before.png	C7 · At testing	2026-11-04 10:28:18	5b3695ed7ed9f7b6a350dee93a53b6e39a0c10c7fab13e31e1b5531030e26129

C. AI usage, costs and human oversight

MODEL	CALLS	INPUT TOKENS	OUTPUT TOKENS	IMAGES	COST (USD)
muse-spark-1.3	15	21,414	122,133	0	0.5458

sam-3.1	7	0	0	7	price not published
Total					**0.5458**

Sensitive data (personal data and secrets) in evidence: 0/18 spotted by Muse Spark + SAM, 18/18 masked in the end thanks to the deterministic safety net (page text).

* sam-3.1: price not published (pricing.toml)

Safeguards

- Scripts: allow-listed imports, no write requests (static analysis before execution).
- Read-only test account: any write would be refused by the system (defence in depth).
- Fixes: allow-listed route and parameters, human approval, separate change account.
- Segregation of duties: the approver cannot be the auditor.

D. Integrity

Audit-trail fingerprint when the report was issued. Verify with `python -m attest verify <folder>`.

38669bddd00027cacbdee5dce4464b9cc493ee4b7c1730216fb17d37dbbd98e2

F. Framework coverage

20 requirements read: 11 tested automatically (7 controls; 4 regulation clauses covered through the policy), 9 left to document review, 0 to interviews, 0 rejected for lack of an exact quote. Attest only covers the first group.

REF.	TEXT (VERIFIED QUOTE)	TESTABILITY	CONTROL	
RTS 16(1)	« financial entities shall develop, document and implement a policy governing the acquisition, development, and maintenance of ICT systems. »	document review	0	Existence and documentation of a governance policy cannot be proven from read-only code alone.
RTS 16(1)(a)	« identify security practices and methodologies relating to the acquisition, development, and maintenance of ICT systems; »	document review	0	Policy content requires document review and cannot be inferred from source code.
RTS 16(1)(c)	« specify measures to mitigate the risk of unintentional alteration or intentional manipulation of the ICT systems during development, maintenance, and deployment. »	document review	0	Specified mitigation measures require policy document review, not code inspection.
RTS 16(2)	« financial entities shall develop, document, and implement an ICT systems' acquisition, development, and maintenance procedure for the testing and approval of all ICT systems prior to their use and after maintenance. »	document review	0	Existence of an approval procedure requires document review beyond the code repository.
RTS 16(3)	« the procedure shall contain source code reviews covering both static and dynamic testing, including security testing for internet-exposed systems and applications. »	tested automatically	→ PDS-2.4	Static security testing in CI is visible via workflow files in the repository interface.
RTS 16(3)(a)	« identify and analyse vulnerabilities and anomalies in the source code; »	document review	0	Vulnerability analysis records and action tracking require document review, not just code.

RTS 16(3)(b)	« adopt an action plan to address those vulnerabilities and anomalies; »	**document review**	0	An action plan is a governance document not visible in the read-only code interface.
RTS 16(3)(c)	« monitor the implementation of that action plan. »	**document review**	0	Monitoring requires follow-up records and cannot be tested via static code alone.
RTS 16(4)	« the procedure shall contain security testing of software packages no later than at the integration phase. »	**tested automatically**	→ PDS-2.5	Integration-phase dependency auditing is visible via CI workflow and requirements files.
RTS 16(5)(a)	« non-production environments only store anonymised, pseudonymised, or randomised production data; »	**tested automatically**	→ PDS-2.6	Test data synthetic marking is visible in versioned test fixtures via the repository interface.
RTS 16(5)(b)	« financial entities are to protect the integrity and confidentiality of data in non-production environments. »	**document review**	0	Environment-level data protection requires architecture and access documents, not just code.
RTS 16(7)	« the procedure shall include controls to protect the integrity of source code of ICT systems that are developed in-house or by an ICT third-party service provider. »	**document review**	0	Source-code integrity controls such as branch protection require platform settings not exposed here.
RTS 16(8)	« proprietary software and, where feasible, the source code provided by ICT third-party service providers, as well as source code coming from open-source projects, are to be analysed and tested in accordance with paragraph »	**tested automatically**	→ PDS-2.5	Dependency pinning and CI auditing of third-party packages are visible in the repository.
PDS-2.1	« Aucun secret (mot de passe, clé, jeton) n'est écrit en clair dans le code source ni dans la configuration versionnée : les secrets viennent d'un coffre ou de variables d'environnement. »	**tested automatically**	1	Hardcoded secrets can be detected by searching versioned code and config via the API.
PDS-2.2	« Tout appel réseau sortant vérifie le certificat TLS du serveur : la vérification n'est jamais désactivée (`verify=False` interdit). »	**tested automatically**	1	Disabled TLS verification patterns are searchable in Python code via the API.
PDS-2.3	« Les mots de passe sont hachés avec argon2, bcrypt, scrypt ou PBKDF2. MD5 et SHA-1 sont interdits pour tout usage de sécurité. »	**tested automatically**	1	Hashing libraries and banned hash functions are searchable in source code.
PDS-2.4	« La chaîne d'intégration continue exécute une analyse statique de sécurité (bandit, semgrep ou CodeQL) à chaque modification. »	**tested automatically**	1	CI workflow files listing static analysis jobs are readable via the repository interface.
PDS-2.5	« Les dépendances sont épinglées à une version exacte (`paquet==version`) et auditées en intégration continue (pip-audit ou équivalent). »	**tested automatically**	1	Requirements pinning and CI audit jobs are readable via tree, file and search.
PDS-2.6	« Les données de test sont synthétiques et déclarées comme telles (champ `"synthetic": true`) ; aucun export de production n'est versionné. »	**tested automatically**	1	Test fixtures and synthetic marking are searchable and readable in tests/ via the API.

PDS-2.7	« Les réussites et les échecs d'authentification sont journalisés par l'application. »	**tested automatically**	1	Application logging calls for authentication events are searchable in source code.